Онтология и вывод

Проектирование по вопросам, машина вывода, SHACL

От интеллектуального поиска к интеллектуальным агентам

Лекция 3

О. М. Атаева А. П. Халов

ВМК МГУ

Осень 2026

AI4Science AI4Research AI4Math

План лекции

1. Задача: что осталось после лекции 2
2. Проектирование по вопросам: онтология, уровни, компетентностные вопросы, три вида классов
3. Качество иерархии: пять ошибок и что находит машина
4. Машина вывода: классификация и объяснения
5. SHACL: проверка данных в замкнутом мире
6. Демонстрация: онтология пересматривается, данные проверяются
7. Архитектура агента: второй верификатор
8. Задание и итоги

Вопросы и принесённые вопросы

Вопросы по статье недели

- Krötzsch, Simančík, Horrocks, «A Description Logic Primer», разделы 1--3.
- Чем ограничение существования отличается от ограничения всеобщности на примере своей области.
- Примеры вопросов: откуда брать классы, если карта знаний нарисована «как видится»? Что делать с требованием, которое проверяется подсчётом? Почему машина вывода молчит о статье с одной рецензией?

Вопросы исследователя из задания лекции 1

- Три-пять вопросов с ожидаемым ответом и источником.
- Сегодня они станут компетентностными вопросами к онтологии.
- Одно требование своей области, которое проверяется подсчётом или сравнением значений.

Установленная библиотека pySHACL понадобится в части 5.

Часть 1

Задача

Вопрос лекции

Как спроектировать онтологию от вопросов исследователя, проверить её машиной вывода и проверить данные на полноту и правильность?

Шаг	Что делаем
задача	онтология лекции 2 записана «как получилось», два требования остались без проверки
проектирование	от вопросов исследователя к классам, свойствам и ограничениям
иерархия	подкласс как частный случай, пять ошибок, что находит машина
машина вывода	классификация определяемых классов, чтение объяснений
SHACL	проверка данных в замкнутом мире: число связей, типы, значения, сравнение
демонстрация	онтология пересматривается, данные проверяются, ответ редактора собирается
агент	проверка данных как второй верификатор

Что осталось от лекции 2

Что получили

- Онтология `review.owl`: 11 классов, 10 свойств, 69 индивидов, правило 6.2.
- Машина вывода: 14 зависимостей пункта 6.5, нарушение 6.2 статьёй P3, противоречие в строке с ошибкой.
- Ответ редактора: статья P3 нарушает требование 6.2.

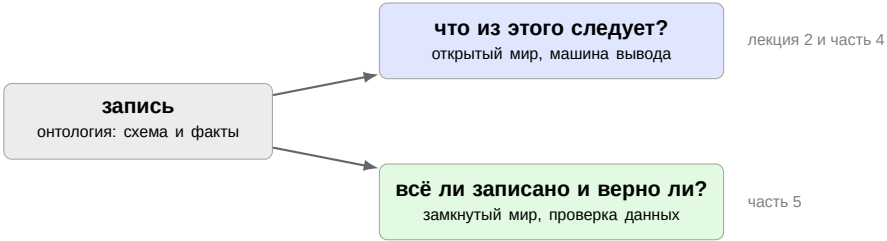
Что осталось открытым

- Классы взяты из карты знаний. Нужны ли они вопросам редактора, никто не проверял.
- Вопрос «какие статьи готовы к решению» онтология не задаёт.
- Требования 6.1 и 6.6 не проверены: у P4 одна рецензия, у P8 ни одной, у P5 рецензия на 27-й день.

Главное

Записать область мало. Онтология должна отвечать на вопросы исследователя, и данные должны проходить проверку.

Два вопроса к одной записи



- На первый вопрос машина вывода отвечает по аксиомам: транзитивность, правило, иерархия. Отсутствие факта она не считает его ложностью.
- На второй вопрос отвечает проверка по форме: сколько связей записано, какого они типа, какие значения. Отсутствие записи здесь есть нарушение.
- Требование 6.1 «не меньше двух рецензий» в открытом мире есть обязательство, в замкнутом мире есть проверка.

Главное

Одна запись, два средства. Машина вывода и проверка данных не заменяют друг друга.

Что получим к концу

	Результат	Чем получен
1	пять вопросов редактора записаны как компетентностные вопросы, у каждого есть средство	методика проектирования
2	онтология пересмотрена: добавлены классы «Сданная рецензия» и «Готовая к решению статья»	три вида классов
3	машина вывода относит к готовым статьи P1, P2, P5, P7 и объясняет, почему P3 туда не попала	классификация, объяснение
4	проверка данных сообщает: P4 и P8 по 6.1, P5 по 6.6, P3 по 6.2, строка с ошибкой	SHACL, замкнутый мир
5	ответ редактора: готовы P1, P2, P7	оба средства вместе

- Машина вывода даёт четыре кандидата. Проверка данных снимает P5 по сроку.
- Требование 6.2 будет записано тремя способами: правило, выражение класса, проверка.

Пример и его запись

Задача редактора

- 8 статей, журнал на 14 рецензий, положение из 31 записи.
- Вопрос: какие статьи выносить на решение и на что опирается требование.

Онтология `review.owl` (лекция 2)

- Классы: Положение с подклассами Раздел, Термин, Требование. Статья, Рецензия, Рецензент, Организация, Утверждение, Доказательство.
- Свойства: `зависит_от` (транзитивное), `на_статью`, `рецензент`, `из_организации`, `организация_авторов`, `нарушает`, свойства данных оценки и дат.
- Правило 6.2, четыре ограничения существования, один определяемый класс.

Четыре требования положения

	Требование	Лекция 2
6.1	не меньше двух сданных рецензий	не проверено
6.2	рецензент не из организации авторов	правило
6.5	есть ответ авторов	не проверено
6.6	рецензия сдана в 21 день	не проверено

Сегодня все четыре требования получают запись и проверку. Скелет прежний: утверждение, обоснование, зависимость.

Часть 2

Проектирование по вопросам

Что такое онтология: два ответа

Т. Грубер: спецификация концептуализации

Онтология есть явная спецификация концептуализации [1]. Позже: формальная явная спецификация разделяемой концептуализации [2]. Записываются согласованные представления сообщества о предметной области.

Б. Смит: представление реальности

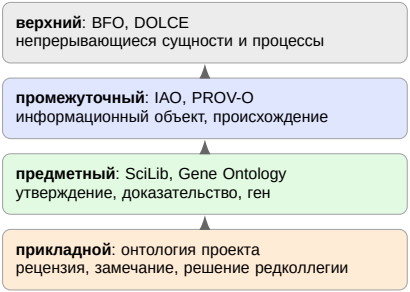
Онтология описывает структуры реальности, которым соответствуют общие термины науки [3]. Термин должен обозначать то, что есть в мире.

	Подход Грубера	Подход Смита
проверка термина	согласован ли в сообществе	есть ли в мире то, что он обозначает
при проектировании	договориться об общем словаре	опереться на онтологию верхнего уровня
в задаче редактора	термины положения: рецензия, замечание	рецензия как информационный объект

Главное

Онтология проекта строится под вопросы исследователя, верхние классы берутся из готовых онтологий.

Уровни онтологий



- Нижние уровни наследуют классы верхних. «Процесс» верхнего уровня подчиняет себе и рецензирование, и численный расчёт.
- BFO есть онтология верхнего уровня, стандарт ISO/IEC 21838-2 [4]. На ней построено больше ста открытых онтологий.
- IAO даёт класс «информационный объект»: статья, рецензия, положение. PROV-O даёт происхождение: кем и из чего получено [5].
- SciLib есть предметная онтология математики: утверждения, доказательства, зависимости. Её классы можно переиспользовать в проекте.

Главное

Онтология проекта начинается с собственных классов и привязывается к верхним уровням там, где это даёт готовые свойства и совместимость с другими графами.

Повторное использование: привязка к верхним классам

Класс примера	Верхний класс
Положение, Статья, Рецензия	`iao:InformationContentEntity`
Рецензент	`prov:Agent`
Организация	`prov:Organization`
Рецензия	`prov:Entity`
источник	`prov:wasDerivedFrom`

Owlready2

```
prov = get_ontology(
    "http://www.w3.org/ns/prov#")
with onto:
    onto.imported_ontologies.append(prov)
    class Рецензент(Объект, prov.Agent):
        pass
    class Организация(Объект,
                      prov.Organization):
        pass
```

Импорт даёт готовые свойства происхождения и совместимость с графами, которые используют те же классы.

- Переиспользуется словарь, принятый сообществом. Собственные классы при этом остаются. Внешние онтологии английские, связь идёт через подкласс: класс примера объявляется подклассом верхнего, русское имя сохраняется.
- Цена: лишняя сложность, если готовые свойства не нужны ни одному вопросу. Привязка делается после того, как вопросы известны.

METHONTOLOGY: от вопроса к онтологии



на всех этапах: сбор знаний, оценка, документирование. Онтология растёт через прототипы

- Методика задаёт жизненный цикл онтологии по образцу программной инженерии [6]. Окончательной онтологии всего нет.
- Потребность формулируется вопросами на естественном языке: что исследователь, сценарий поиска или агент должны узнать из графа.
- В лекции 2 пройдены концептуализация и реализация. Сегодня добавляются спецификация, оценка и проверка данных.

Главное

Первый шаг методики есть список вопросов. Классы и свойства следуют за ним.

Компетентностные вопросы

Компетентностный вопрос есть вопрос на естественном языке, на который онтология с фактами должна давать ответ [7]. Набор вопросов задаёт объём онтологии и служит критерием её проверки.

Что делает вопрос компетентностным

- У него есть ожидаемый ответ и источник, по которому ответ проверяется.
- Ответ выражается средствами онтологии: классом, путём, запросом, проверкой.
- Каждый класс и каждое свойство нужны хотя бы одному вопросу.

Пять вопросов исследователя из задания лекции 1

- Типы «факт», «зависимость», «смысл» были обязательны.
- У каждого записаны ожидаемый ответ и источник.
- Это и есть компетентностные вопросы к вашей онтологии. Сегодня они получают запись.

Главное

Вопрос, у которого есть ожидаемый ответ, есть тест. Онтология проверяется тем, отвечает ли она на свои вопросы.

Пять вопросов редактора

	Вопрос	Ожидаемый ответ	Средство
1	Какие статьи готовы к решению редколлегии?	P1, P2, P7	выводимый класс и проверка данных
2	На какие положения опирается требование 6.5?	14 положений	транзитивное свойство
3	У каких статей рецензент из организации авторов?	P3	правило или проверка
4	Кто рецензировал статьи организации МГУ?	Rv1, Rv2, Rv3, Rv6	запрос (лекция 4)
5	Есть ли в положении норма о сроке без слова «срок»?	требование 6.6	поиск по смыслу (лекция 6)

- У каждого вопроса своё средство. Онтология отвечает на первые три, граф и поиск на остальные.
- Вопрос 1 требует классов, которых в онтологии лекции 2 нет. Отсюда пересмотр.

Три вида классов

Вид	Аксиома	Что даёт
простой место в иерархии	Требование $\sqsubseteq$ Положение	наследование: всё, что верно для положения, верно для требования
связанный необходимое условие со связью	Рецензия $\sqsubseteq$ $\exists$на_статью.Статья	обязательство: связь есть, даже если не записана
выводимый необходимое и достаточное условие	СданнаяРецензия $\equiv$ Рецензия $\sqcap$ $\exists$сдано	принадлежность вычисляет машина

- В лекции 2 были все три вида: иерархия положений, ограничения существования, класс «Базовое положение».
- Компетентностный вопрос вида «какие объекты удовлетворяют условию» записывается выводимым классом.

Главное

Простые классы задают иерархию, связанные задают обязательства, выводимые задают вопросы, на которые машина отвечает сама.

От вопроса к выражению класса

Вопрос 1. Какие статьи готовы к решению редколлегии?

Положение. Не меньше двух сданных рецензий (6.1). Рецензент не из организации авторов (6.2). Есть ответ авторов (6.5). Рецензии сданы в срок (6.6).

$$\text{ГотоваяКРешениюСтатья} \equiv \text{Статья} \sqcap \geq\ 2\ \text{имеет_рецензию.СданнаяРецензия} \sqcap \forall \text{имеет_рецензию.(Рецензия} \sqcap \text{ответ_авторов} = \text{да)} \sqcap \neg\exists \text{нарушает.Требование}$$

- Три условия из четырёх записаны выражением класса. Срок (6.6) требует сравнения дат, в выражение класса он не входит.
- Выражение читается как требование положения. Это и есть проверка понимания: если выражение не читается, вопрос записан неверно.
- Машина вывода отнесёт статью к классу только тогда, когда все условия следуют из записи. В открытом мире для этого нужно замыкание.

Выражения для остальных вопросов

	Вопрос	Запись
2	на что опирается 6.5	значение транзитивного свойства: R65.зависит_от после вывода
3	рецензент из организации авторов	правило лекции 2. Как класс: Статья $\sqcap$ $\exists$нарушает.{R62}. Как проверка: форма SHACL в части 5
4	кто рецензировал статьи МГУ	Рецензент $\sqcap$ $\exists$рецензент$^{-}$.($\exists$на_статью.($\exists$организация_авторов.{МГУ})). Удобнее запросом SPARQL
5	норма о сроке без слова «срок»	онтологией не записывается. Ответ даёт поиск по смыслу, классы помогают сузить выдачу до требований

- R^{-} есть обратное свойство, $\{a\}$ есть класс из одного индивида (буквы $\mathcal{I}$ и $\mathcal{O}$ в SROIQ).
- У каждого выражения есть денотат: множество индивидов, которое и есть ответ.

Главное

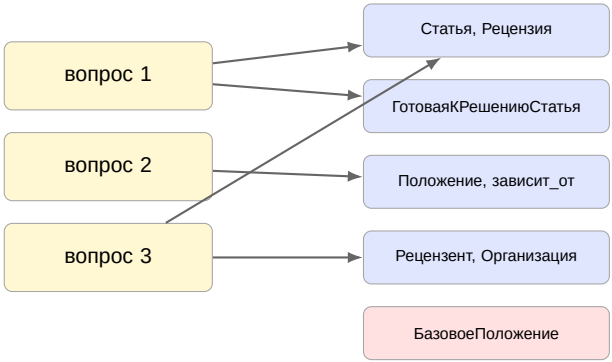
Вопрос, который не выражается ни классом, ни путём, ни проверкой, записывается как задача для другого инструмента. Онтология при этом даёт ему типы.

Пересмотр онтологии лекции 2

Что	Почему	Вопрос
добавить класс СданнаяРецензия	6.1 считает только сданные рецензии	1
добавить класс ГотоваяКРешениюСтатья	ответ на вопрос редактора	1
добавить свойство данных `срок`	6.6 сравнивает дату сдачи со сроком	1, проверка
оставить Положение, Раздел, Термин, Требование, `зависит_от`	зависимости и источники	2, 3
оставить Утверждение, Доказательство	вопросы к статье P2	вариант по математике
проверить БазовоеПоложение	ни один вопрос редактора его не спрашивает	кандидат на удаление

- Пересмотр идёт от списка вопросов к списку классов, в обе стороны.
- Класс без вопроса удаляется или получает вопрос. Вопрос без класса получает класс, путь или проверку.

Критерий: каждый класс нужен вопросу



- Таблица «вопрос — класс — свойство» заполняется до записи онтологии и проверяется после.
- Пустая строка в таблице означает лишний класс. Пустой столбец означает вопрос без ответа.

Главное

Объём онтологии задают вопросы. Это защита и от недостатка классов, и от избытка.

Рецепт: от карты знаний к онтологии

1. **Вопросы**: три-пять компетентностных вопросов с ожидаемым ответом и источником.
2. **Карта знаний**: понятия и связи, как их видит исследователь (задание лекции 1).
3. **Класс или индивид**: для каждого узла карты, это множество объектов или один объект.
4. **Иерархия**: какие классы являются частными случаями других, какие не пересекаются.
5. **Свойства**: имя, область определения, область значений, характеристики.
6. **Аксиомы**: ограничения существования и всеобщности, выводимые классы для вопросов вида «какие объекты».
7. **Проверка**: таблица «вопрос — класс», запуск машины вывода, формы SHACL для требований к данным.

- В лекции 2 рецепт начинался с шага 2. Сегодня он начинается с шага 1 и заканчивается шагом 7.
- Задание лекции 3 проходит рецепт на своей онтологии версии 1.

Главное

Онтология проекта начинается с вопросов исследователя и растёт вместе с проектом.

Часть 3

Качество иерархии

Подкласс есть частный случай

$A \sqsubseteq B$ означает: каждый экземпляр A есть экземпляр B в любом состоянии мира.

Проверка фразой

- «Каждое требование является положением». Верно.
- «Каждая рецензия является статьёй». Неверно. Это связь, не подкласс.
- «Каждый рецензент является человеком». Верно, пока рецензент есть человек.

Что наследуется вниз

- Ограничения: если у каждого положения есть источник, он есть у каждого требования.
- Области свойств: свойство, заданное на положении, применимо к требованию.
- Принадлежность к верхним классам: требование есть объект.

Ошибка в иерархии наследуется вместе с остальным.

Главное

Проверка фразой «каждый A является B» занимает секунду и ловит большинство ошибок иерархии.

Ошибка 1. Часть вместо подкласса

Запись: Раздел $\sqsubseteq$ Документ, Пункт $\sqsubseteq$ Раздел.

Проверка: «каждый раздел является документом». Неверно. Раздел есть часть документа.

Как правильно: свойство `часть_документа` с областями Раздел $\rightarrow$ Документ. Отношение части транзитивно, как и зависимость.

Оговорка о нашей онтологии
В лекции 2 записано Раздел $\sqsubseteq$ Положение. Там «Положение» означает запись положения, то есть строку корпуса. «Каждая запись-раздел является записью положения» верно.

Если бы «Положение» означало документ целиком, запись была бы этой ошибкой.

Главное

Имя класса должно называть то, чем являются его экземпляры. Двусмысленное имя порождает ошибку иерархии.

Ошибка 2. Роль вместо типа

Запись: Рецензент $\sqsubseteq$ Человек, Автор $\sqsubseteq$ Человек, Рецензент и Автор не пересекаются.

Что ломается: Rv1 рецензирует статью P1 и пишет статью P9. Он оказывается в двух непересекающихся классах. Противоречие.

Причина: «рецензент» есть роль человека в отношении к статье. Роль появляется и исчезает, тип остаётся.

Как правильно

- Класс Человек. Роли записываются свойствами: `рецензент`(рецензия, человек), `автор`(статья, человек).
- Или классы ролей без дизъюнктности: Рецензент $\equiv$ Человек $\sqcap$ $\exists$рецензент$^{-}$.Рецензия.

В онтологии лекции 2 класс Рецензент оставлен: у примера нет авторов как людей, и ошибка не проявляется. В своей онтологии проверяйте роли.

Главное

OntoClean одной фразой: жёсткий класс объект не может покинуть (Человек), роль может (Рецензент). Роли не объявляются непересекающимися [8].

Ошибки 3 и 4. Индивид на месте класса, класс по значению признака

Ошибка 3: R65 $\sqsubseteq$ Требование.

- «Каждый R65 является требованием» бессмысленно: R65 один.
- Машина не возражает: пустой класс допустим. Ошибка видна только по смыслу и по тому, что у R65 нет фактов.
- Правильно: R65 есть индивид класса Требование.

Ошибка 4: СтатьяМГУ, СтатьяМФТИ, СтатьяИТМО как классы.

- Класс на каждое значение признака. Новая организация требует нового класса. Схема зависит от данных.
- Правильно: свойство `организация_авторов` и, если нужен класс, выводимый: Статья $\sqcap$ $\exists$организация_авторов.{МГУ}.

Обе ошибки встречаются в первых онтологиях чаще остальных. Скрипт проверки задания лекции 2 ловит третью. Четвёртая видна по именам классов.

Ошибка 5. Множественное наследование без основания

Запись: Рецензия $\sqsubseteq$ Документ, Рецензия $\sqsubseteq$ Событие.

Проверка: «каждая рецензия является документом» верно. «Каждая рецензия является событием» верно для рецензирования, неверно для текста рецензии.

Одним словом названы две вещи: текст и процесс. Верхние онтологии их разделяют: информационный объект и процесс не пересекаются.

Как правильно

- Два класса: Рецензия (текст) и Рецензирование (процесс), связь `результат`.
- Множественное наследование допустимо, когда оба надкласса верны для каждого экземпляра: Теорема $\sqsubseteq$ Утверждение, Теорема $\sqsubseteq$ ДоказанноеУтверждение.

Привязка к BFO или IAO сразу показывает такие склейки: класс не может быть подклассом двух непересекающихся верхних.

Главное

Пять ошибок: часть вместо подкласса, роль вместо типа, индивид вместо класса, класс по значению признака, склейка двух сущностей в одном классе.

Дизъюнктность и неудовлетворимый класс

Запись: Статья и Положение не пересекаются. Добавлен класс СтатьяПоложение $\sqsubseteq$ Статья $\sqcap$ Положение.

Машина вывода: класс эквивалентен $\bot$ (Nothing). У него не может быть ни одного экземпляра.

Это проверка схемы, не данных. Противоречия в записи нет, пока в класс не попал индивид.

Что ещё находит машина в схеме

- **Выведенная иерархия**: СданнаяРецензия $\sqsubseteq$ Рецензия получается из определения, записывать не нужно.
- **Эквивалентные классы**: два определения с одним денотатом. Один из них лишний.
- **Неудовлетворимые классы**: ошибка в аксиомах, чаще всего дизъюнктность и ограничение всеобщности.

Снимок окна Protégé: приложение Д.

Главное

Запуск машины вывода на схеме без данных есть первая проверка онтологии. Неудовлетворимый класс есть ошибка проектирования.

Проверка иерархии: порядок действий

Шаг	Что сделать	Что ловит
фраза	«каждый A является B» для каждой связи подкласса	ошибки 1, 2, 5
имена	индивид или класс, значение признака или свойство	ошибки 3, 4
дизъюнктность	объявить непересекающимися верхние классы	склейки, противоречия в данных
машина вывода на схеме	неудовлетворимые классы, выведенная иерархия	ошибки аксиом
таблица вопросов	каждый класс нужен вопросу	лишние классы
скрипт задания	формальные критерии	индивид на месте класса, плоская иерархия

- Шаги занимают минуты и выполняются до наполнения онтологии данными.
- Задание лекции 3 требует записать результат проверки в описание модели.

Классы онтологии и типы в программе

Класс онтологии

- Множество индивидов, заданное именем, иерархией и аксиомами.
- Принадлежность выводится: индивид может оказаться в классе, о котором автор не писал.
- Открытый мир: отсутствие записи не есть отрицание.

Тип в языке программирования

- Множество значений, проверяемое до выполнения.
- Принадлежность объявляется: значение имеет тот тип, который написан.
- Замкнутый мир: чего нет в объявлении, того нет.

- Во второй части курса классы онтологии станут типами для внешней проверки: утверждение, доказательство, зависимость получат типы в языке с проверкой доказательств. Математика здесь служит примером.
- Проверка данных по форме (часть 5) стоит между ними: формы проверяются как типы, записываются как классы.

Главное

Онтология описывает, что может быть. Тип и форма проверяют, что есть.

Часть 4

Машина вывода: классификация и объяснения

Три задачи машины вывода

Задача	На примере	Где в курсе
вывод неявных фактов	6.5 зависит от раздела 1, P3 нарушает 6.2	лекция 2: транзитивность, правило
проверка непротиворечивости	организация в поле «рецензент»	лекция 2: область значений и дизъюнктность
классификация	СтатьяПоложение $\equiv \bot$. P1 есть готовая к решению статья	**сегодня**: схема и индивиды

- Классификация схемы: машина строит выведенную иерархию классов и находит неудовлетворимые.
- Классификация индивидов: машина относит индивидов к выводимым классам по их определениям.
- Объяснение: для каждого вывода машина называет аксиомы и факты, из которых он получен.

Главное

Машина вывода ничего не знает о рецензировании. Смысл задают аксиомы, и сегодня аксиомы заданы вопросами редактора.

Классификация схемы

```
with onto:
    class СданнаяРецензия(Рецензия):
        equivalent_to = [Рецензия & сдано.min(1, date)]
    class ГотоваяКРешениюСтатья(Статья):
        equivalent_to = [Статья
            & имеет_рецензию.min(2, СданнаяРецензия)
            & имеет_рецензию.only(Рецензия
                  & ответ_авторов.value(True))
            & Not(нарушает.some(Требование))]
    class СтатьяПоложение(Статья, Положение): pass

sync_reasoner_pellet(world)
СтатьяПоложение.equivalent_to   # [owl.Nothing]
```

Что машина сделала со схемой

- Поставила СданнаяРецензия под Рецензия: следует из определения.
- Поставила ГотоваяКРешениюСтатья под Статья.
- Нашла, что СтатьяПоложение не может иметь экземпляров: классы Статья и Положение объявлены непересекающимися.

Неудовлетворимый класс сообщается до того, как в онтологию попали данные.

Главное

Классификация схемы есть проверка проектирования. Она выполняется после каждого изменения аксиом.

Классификация индивидов: открытый мир

После первого запуска

- СданнаяРецензия: 13 индивидов. У рецензии P8 даты сдачи нет, она не попала.
- ГотоваяКРешениюСтатья: **пусто**.

У статьи P1 записаны две сданные рецензии с ответом авторов и нет нарушений. Почему она не в классе?

Открытый мир

- Условие ∀имеет_рецензию.(…ответ авторов) требует, чтобы **все** рецензии статьи имели ответ. Машина допускает, что есть ещё рецензии, о которых не записано.
- Условие ¬∃нарушает требует отсутствия нарушений. Машина допускает незаписанное нарушение.

Всеобщность и отрицание в открытом мире недоказуемы по записи.

Главное

Выводимый класс с условиями «все» и «нет» наполняется только после сообщения о полноте записи.

Замыкание мира

```
with onto:
    for a in Статья.instances():
        close_world(a, Properties=[имеет_рецензию,
                                   нарушает])
sync_reasoner_pellet(world)

ГотоваяКРешениюСтатья.instances()
# [P1, P2, P5, P7]
```

`close_world` добавляет аксиомы: у статьи ровно те рецензии, что записаны, и ровно те нарушения, что выведены.

Четыре кандидата

- P1, P2, P7: две сданные рецензии, ответ авторов, нарушений нет.
- P5: то же самое. Рецензия Rv3 сдана на 27-й день, но в выражении класса срока нет.
- P3 исключена: выведено нарушение 6.2.
- P4, P8: меньше двух сданных рецензий. P6: нет ответа авторов.

Замыкание есть явное заявление «записано всё». Оно делается осознанно и записывается в журнал.

Главное

Машина вывода с замыканием мира отвечает на вопрос 1 с точностью до срока. Сравнение дат остаётся проверке данных.

Чтение объяснения: почему P3 не готова

```
Почему P3 не является ГотоваяКРешениюСтатья:
  P3 нарушает R62                  [выведено]
    из правила 6.2:
      рец_P3_Rv4 рецензент Rv4
      Rv4 из_организации ФИЦ_ИУ
      P3 организация_авторов ФИЦ_ИУ
  ГотоваяКРешениюСтатья ▫ ¬▫нарушает.Требование
  R62 type Требование
```

Объяснение собрано из вывода машины: нарушение получено правилом, правило опирается на три факта журнала, определение класса исключает статьи с нарушением.

Как читать объяснение

1. Найти факт, который сделал вывод: P3 нарушает R62.
2. Найти, откуда он: правило и три факта.
3. Найти аксиому, которая связывает факт с классом.

Объяснение прослеживается до строк журнала и пункта положения. Это второй и третий уровни объяснения из лекции 1: правило и свидетельство.

Чтение объяснения: противоречие

```
Explanation(s):
1) DisjointClasses(... Организация ... Рецензент ...)
   рец_P9_ошибка рецензент ФИЦ_ИУ
   рецензент range Рецензент
   ФИЦ_ИУ type Организация
```

Четыре строки, четыре участника: аксиома дизъюнктности, факт из строки, область значений, тип индивида. Каждая строка по отдельности допустима.

Что делать с объяснением

- Найти строку, которая пришла из данных: факт о рецензенте. Остальные строки есть схема.
- Исправить данные, если схема верна. Исправить схему, если данные верны.
- Здесь верна схема: в поле «рецензент» записана организация.

Машина называет участников противоречия. Решение о том, кто виноват, принимает человек.

Снимок окна Protégé: приложение Д.

Снимок окна Protégé: приложение Д.

Главное

Объяснение превращает вывод машины в проверяемую цепочку. Без объяснения вывод остаётся чёрным ящиком.

Чего машина вывода не проверяет

Вопрос	Машина вывода	Нужно
у статьи P4 одна рецензия, норма две	обязательство: вторая рецензия есть, пока не названа	подсчёт записанных связей
у P5 рецензия сдана на 27-й день	значения дат не сравнивает	сравнение значений
в поле «оценка» строка вместо числа	тип данных нарушен, сообщение об ошибке данных	проверка типа
шкала 7 вместо 5 или 10	допустимо, пока не записано иначе	допустимые значения
у рецензии нет статьи	обязательство: статья есть, пока не названа	обязательное поле

- Все пять вопросов относятся к записи, не к следствиям из неё. Это вопросы замкнутого мира.
- Замыкание мира через `close_world` решает первый из них ценой новых аксиом на каждый индивид. Остальные не решает.

Главное

Для вопросов «всё ли записано и верно ли» нужно отдельное средство. Оно называется SHACL.

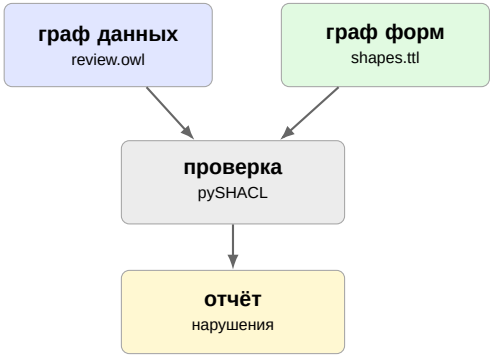
Часть 5

SHACL: проверка данных в замкнутом мире

Второе средство

SHACL, Shapes Constraint Language, есть стандарт W3C для проверки графов данных по формам [9].

- **Граф данных**: наши индивиды и факты. Тот же файл, что читает машина вывода.
- **Граф форм**: требования к данным. Отдельный файл.
- **Отчёт**: список нарушений, у каждого узел, свойство, сообщение.



- Проверка идёт в замкнутом мире: считается только то, что записано. Выведенное машиной к записанному не относится.
- Форма записывается на том же языке, что данные: тройками, в Turtle.

Главное

Онтология говорит, что может быть. Форма говорит, что должно быть записано.

Узловая форма и её цель

```
@prefix sh:  <http://www.w3.org/ns/shacl#> .
@prefix :    <http://example.org/review.owl#> .
@prefix shp: <http://example.org/shapes#> .

shp:Рецензия a sh:NodeShape ;
    sh:targetClass :Рецензия ;
    sh:property [ ... ] ;
    sh:property [ ... ] .
```

- `sh:NodeShape`: форма для узлов графа.
- `sh:targetClass`: к каким узлам она применяется. Ко всем индивидам класса Рецензия.
- `sh:property`: требования к свойствам узла, по одному блоку на свойство.

Форма есть набор требований положения к одной записи журнала.

- Цель задаётся и иначе: `sh:targetNode` для одного узла, `sh:targetSubjectsOf` для всех субъектов свойства.
- Форма и класс онтологии носят разные имена, чтобы их не путать: класс `:Рецензия`, форма `shp:Рецензия`.

Свойство формы: путь и число связей

```
shp:Рецензия a sh:NodeShape ;
  sh:targetClass :Рецензия ;
  sh:property [
    sh:path :на_статью ;
    sh:minCount 1 ;
    sh:maxCount 1 ;
    sh:message "у рецензии ровно одна статья" ] .
```

- `sh:path`: свойство, которое проверяется.
- `sh:minCount`, `sh:maxCount`: сколько значений должно быть записано.
- `sh:message`: текст для отчёта.

Та же мысль	В OWL	В SHACL
у каждой рецензии есть статья	Рецензия $\sqsubseteq$ $\exists$на_статью.Статья	`sh:minCount 1`
у рецензии одна статья	`на_статью` функциональное	`sh:maxCount 1`

Главное

OWL из той же фразы выводит, что статья есть. SHACL проверяет, что она записана.

Тип значения и класс

```
sh:property [
    sh:path :рецензент ;
    sh:class :Рецензент ;
    sh:minCount 1 ; sh:maxCount 1 ;
    sh:message "рецензентом должен быть рецензент" ] ;
sh:property [
    sh:path :оценка ;
    sh:datatype xsd:decimal ;
    sh:minInclusive 0 ;
    sh:message "оценка есть число" ] .
```

- `sh:class`: значение должно быть индивидом класса. Проверяется запись типа, иерархия классов учитывается по `rdfs:subClassOf`.
- `sh:datatype`: тип литерала.
- `sh:minInclusive`, `sh:maxInclusive`: границы числа или даты.

- Строка с организацией в поле «рецензент» нарушает `sh:class`. Машина вывода на ней выводила противоречие из четырёх аксиом. Форма сообщает сразу: значение не того класса.
- Область значений в OWL выводит тип. `sh:class` в SHACL проверяет тип. Одна фраза положения, два разных действия.

Допустимые значения и сравнение

Ядро SHACL: свойство `срок` вычислено при загрузке

```
sh:property [
    sh:path :шкала ;
    sh:in (5 10) ;
    sh:message "шкала 5 или 10" ] ;
sh:property [
    sh:path :сдано ;
    sh:lessThanOrEquals :срок ;
    sh:message "6.6: рецензия сдана позже срока" ] .
```

Срок есть дата назначения плюс 21 день. Вычисление сделано в коде при загрузке, проверка записана в данных.

Тот же срок внутри запроса

```
sh:sparql [
  sh:message "6.6: рецензия сдана позже срока" ;
  sh:select """
    SELECT $this WHERE {
      $this :назначено ?a ; :сдано ?s .
      FILTER (?s > ?a +
        "P21D"^^xsd:dayTimeDuration) }
  """ ] .
```

Без свойства `срок`: арифметика дат есть в SPARQL. Ядро SHACL её не содержит.

- `sh:in` задаёт список допустимых значений, `sh:pattern` регулярное выражение для строк. `sh:lessThan`, `sh:lessThanOrEquals`, `sh:equals`, `sh:disjoint` сравнивают значения двух свойств одного узла.
- Первая ошибка агента из лекции 1, усреднение по разным шкалам, здесь ловится формой: шкала записана и проверена.

Главное

Сравнение значений двух свойств есть то, чего не было в OWL. Требование о датах записывается двумя способами: сравнением готовых значений в ядре или запросом с арифметикой.

Путь и обратный путь

```
shp:Статья a sh:NodeShape ;
  sh:targetClass :Статья ;
  sh:property [
    sh:path [ sh:inversePath :на_статью ] ;
    sh:minCount 2 ;
    sh:message "6.1: рецензий меньше двух" ] .
```

`sh:inversePath`: свойство в обратную сторону. От статьи к рецензиям, которые на неё ссылаются.

Почему не `имеет_рецензию`

- В онтологии `имеет_рецензию` объявлено обратным к `на_статью`. Машина вывода его выводит.
- В файле записано только `на_статью`: 14 троек. Троек `имеет_рецензию` ноль.
- Форма с `sh:path :имеет_рецензию` нашла бы нарушение у всех восьми статей.

- Пути составляются: `sh:path (:на_статью :организация_авторов)` есть путь из двух шагов. Есть и `sh:zeroOrMorePath` для транзитивных цепочек.

Главное

SHACL видит только записанное. Выведенные свойства, обратные и транзитивные, в форме задаются путём.

Составная форма: требование 6.1

```
shp:Статья a sh:NodeShape ;
  sh:targetClass :Статья ;
  sh:property [
    sh:path [ sh:inversePath :на_статью ] ;
    sh:qualifiedValueShape [
        sh:property [ sh:path :сдано ; sh:minCount 1 ] ] ;
    sh:qualifiedMinCount 2 ;
    sh:message "6.1: сданных рецензий меньше двух" ] .
```

- Положение считает **сданные** рецензии: две рецензии, у которых есть дата сдачи.
- `sh:qualifiedValueShape`: форма, которой должны удовлетворять считаемые значения.
- `sh:qualifiedMinCount`: сколько таких значений нужно.

- Результат на журнале: P4 (одна сданная рецензия), P8 (ни одной). Остальные шесть статей проходят.
- Та же мысль в OWL: ≥ 2 имеет_рецензию.СданнаяРецензия. Там она есть обязательство, здесь есть проверка.

Когда ядра не хватает: требование 6.2

```
shp:Рецензия sh:sparql [
  sh:message "6.2: рецензент из организации авторов" ;
  sh:select """
    SELECT $this WHERE {
      $this :рецензент ?p .
      ?p :из_организации ?o .
      $this :на_статью ?s .
      ?s :организация_авторов ?o . }
  """ ] .
```

Запрос находит рецензии, у которых организация рецензента совпадает с организацией авторов. Каждая найденная рецензия есть нарушение.

- Ядро SHACL сравнивает значения двух свойств одного узла. Здесь сравниваются значения на концах двух путей.
- `sh:sparql`: ограничение запросом. Переменная `$this` есть проверяемый узел.
- Язык запросов SPARQL целиком в лекции 4. Здесь достаточно прочитать четыре строки как четыре условия правила.

Главное

Те же четыре условия, что в правиле лекции 2. Правило выводит факт «нарушает». Форма сообщает о нарушении.

Требование 6.2 в трёх записях

Запись	Как выглядит	Что даёт	Где
правило SWRL	пять условий → нарушает(s, R62)	новый факт в графе, виден машине вывода и агенту	лекция 2
выражение класса	Статья ⊓ ∃нарушает.{R62}	класс, по которому можно спрашивать	часть 2
форма SHACL с запросом	`sh:sparql` с теми же условиями	нарушение в отчёте с сообщением	часть 5
запрос SPARQL	`SELECT ?s WHERE` с теми же условиями	список статей по требованию	лекция 4

- Условия во всех записях одни. Различается, что происходит с результатом: факт, класс, нарушение, список.
- Правило меняет граф. Форма не меняет: она только сообщает. Для проверки данных это преимущество.

Главное

Одно требование положения, четыре записи. Выбор записи определяется тем, что нужно дальше: выводить, спрашивать, проверять или перечислять.

Отчёт о проверке

```
Validation Report
Conforms: False
Results (5):
Constraint Violation in QualifiedMinCountConstraintComponent
    Focus Node: :P4
    Result Path: [ sh:inversePath :на_статью ]
    Message: 6.1: сданных рецензий меньше двух
Constraint Violation in LessThanOrEqualsConstraintComponent
    Focus Node: :рец_P5_Rv3
    Result Path: :сдано
    Message: 6.6: рецензия сдана позже срока
Constraint Violation in SPARQLConstraintComponent
    Focus Node: :рец_P3_Rv4
    Message: 6.2: рецензент из организации авторов
```

Как читать результат

- `Conforms`: прошли ли данные проверку целиком.
- `Focus Node`: узел, в котором нарушение.
- `Result Path`: свойство, по которому оно найдено.
- `Message`: текст из формы. Он должен называть пункт положения.

Отчёт сам есть граф: его можно сохранить, запросить и отдать агенту.

Строгость и запуск

Строгость нарушения

```
sh:property [
    sh:path :ответ_авторов ;
    sh:minCount 1 ;
    sh:severity sh:Warning ;
    sh:message "6.5: ответ авторов не записан" ] .
```

- `sh:Violation`: данные не принимаются. По умолчанию.
- `sh:Warning`: данные принимаются, нарушение в отчёте.
- `sh:Info`: замечание.

Запуск

```
pyshacl -s kg/shapes.ttl kg/ontology.ttl

from pyshacl import validate
ok, report, text = validate(data, shacl_graph=shapes)
```

- Проверка выполняется без машины вывода: `inference="none"`. Так форма видит ровно то, что записано.
- Можно включить вывод RDFS или OWL RL перед проверкой. Тогда обратные свойства появятся, и граница записанного сдвинется. Решение записывается в журнал.

SHACL и OWL рядом

	OWL и машина вывода	SHACL и проверка
мир	открытый	замкнутый
отсутствие факта	ничего не означает	нарушение, если факт обязателен
фраза «у статьи две рецензии»	обязательство: вторая есть, пока не названа	проверка: записано ли две
область значений	выводит тип значения	проверяет тип значения
результат	новые факты, классы, противоречие	отчёт о нарушениях
граф данных	меняется: добавляются выведенные тройки	не меняется
вопрос	что следует из записанного	всё ли записано и верно ли

- Одна онтология, две системы правил поверх неё. Классы и свойства общие.
- Порядок в курсе: сначала вывод, потом проверка. Выведенные факты можно записать в граф и проверить формой.

Главное

Машина вывода и проверка данных отвечают на разные вопросы к одной записи. В проекте нужны оба.

Что проверяет форма и что проверяет человек

Форма проверяет

- Полноту записи: обязательные поля, число связей.
- Типы и допустимые значения.
- Соотношения значений: даты, числа, совпадения.
- Структуру: у каждого утверждения есть доказательство, у каждой теоремы есть зависимость.

Форма не проверяет

- Верно ли содержание: доказательство записано, но верно ли оно. Это внешняя проверка, часть II.
- Полон ли сам журнал: форма видит записанные статьи. Статью, которой нет в журнале, она не найдёт.
- Смысл оценок и замечаний. Это суждение редактора.

Отчёт проверки есть свидетельство третьего уровня объяснения: строки, на которых нарушение найдено.

Главное

Проверка по форме отвечает за форму записи. За содержание отвечают внешняя проверка и человек.

Проверка данных в SciLib

- Граф знаний SciLib содержит около 66 млн троек. Данные поступают из разных источников: извлечение из текстов, импорт из библиографических баз, ручная разметка.
- Перед записью в граф данные проходят проверку по формам: у каждого утверждения есть источник, у каждой публикации есть авторы и год, идентификаторы имеют заданный вид, даты согласованы.
- Нарушения не попадают в граф. Они возвращаются источнику данных с отчётом, как отчёт SHACL.
- Тот же подход в задании: граф проекта принимается, когда проверка проходит. Скрипт проверки задания запускает pySHACL.

Главное

В действующей системе проверка по формам стоит на входе в граф. Это та же роль, что у верификатора в агенте.

Часть 6

Демонстрация

Сценарий

Блокнот `demo_lecture3.ipynb`. Данные: онтология `review.owl` лекции 2, журнал `reviews.csv`, формы `shapes.ttl`.

Шаг	Что на экране	На что смотреть
1	пять вопросов редактора и их средства	у каждого вопроса ожидаемый ответ
2	пересмотр: два выводимых класса и свойство срок	три записи Owlready2
3	машина вывода: класс пуст, замыкание, четыре кандидата	открытый мир
4	объяснение: почему P3 и P6 не готовы	чтение объяснения
5	неудовлетворимый класс в схеме	проверка проектирования
6	граф форм: пять форм для положения	форма читается как требование
7	проверка: отчёт с пятью нарушениями	замкнутый мир
8	строка с ошибкой: форма и машина вывода рядом	два сообщения об одной ошибке
9	ответ редактора: P1, P2, P7	зачем оба средства

Шаг 1. Вопросы и средства

```
ВОПРОСЫ = [
 ("Какие статьи готовы к решению редколлегии?",      "выводимый класс + проверка", "P1, P2, P7"),
 ("На какие положения опирается требование 6.5?",   "транзитивное свойство",      "14 положений"),
 ("У каких статей рецензент из организации авторов?", "правило или проверка",       "P3"),
 ("Кто рецензировал статьи организации МГУ?",        "запрос (лекция 4)",          "Rv1, Rv2, Rv3, Rv6"),
 ("Есть ли норма о сроке без слова «срок»?",         "поиск по смыслу (лекция 6)", "6.6"),
]
```

- Вопросы взяты из задания лекции 1: тип, ожидаемый ответ, источник.
- Столбец «средство» заполняется до записи онтологии. Он говорит, что нужно добавить.

Шаг 2. Пересмотр онтологии

```
with onto:
    class срок(DataProperty, FunctionalProperty):
        domain = [Рецензия]; range = [datetime.date]
    for r in Рецензия.instances():
        r.срок = r.назначено + timedelta(days=21)

    class СданнаяРецензия(Рецензия):
        equivalent_to = [Рецензия & сдано.min(1, date)]
    class ГотоваяКРешениюСтатья(Статья):
        equivalent_to = [Статья
            & имеет_рецензию.min(2, СданнаяРецензия)
            & имеет_рецензию.only(Рецензия
                  & ответ_авторов.value(True))
            & Not(нарушает.some(Требование))]
```

- Свойство срок вычислено из даты назначения: положение даёт 21 день.
- Два выводимых класса записаны по вопросу 1.
- Остальная онтология лекции 2 не изменилась.

Пересмотр занял три записи. Их основание записано в таблице «вопрос — класс».

Шаг 3. Машина вывода и замыкание

```
sync_reasoner_pellet(world, infer_property_values=True)
СданнаяРецензия: 13 индивидов
ГотоваяКРешениюСтатья до замыкания: []

for a in Статья.instances():
    close_world(a, Properties=[имеет_рецензию, нарушает])
sync_reasoner_pellet(world, infer_property_values=True)
ГотоваяКРешениюСтатья после замыкания: ['P1', 'P2', 'P5', 'P7']
```

- До замыкания класс пуст: условия «все рецензии с ответом» и «нет нарушений» в открытом мире не доказуемы.
- После замыкания четыре кандидата. P3 исключена правилом 6.2, P6 ответом авторов, P4 и P8 числом рецензий.
- P5 в списке: срок не входит в выражение класса.

Главное

Машина вывода ответила на вопрос 1 с точностью до одного требования. Это требование о значениях дат.

Шаг 4. Объяснения

```
почему Р3 не готова:
  Р3 нарушает R62  <- правило 6.2: рец_Р3_Rv4 рецензент Rv4, Rv4 из_организации ФИЦ_ИУ,
                                   Р3 организация_авторов ФИЦ_ИУ
  ГотоваяКРешениюСтатья ▫ ¬▫нарушает.Требование
почему Р6 не готова:
  рец_Р6_Rv4 ответ_авторов false, рец_Р6_Rv5 ответ_авторов false
  ГотоваяКРешениюСтатья ▫ ▫имеет_рецензию.(ответ_авторов = true)
почему Р4 не готова:
  сданных рецензий 1, требуется не меньше двух
```

- Объяснение собирается из фактов журнала и аксиом определения. В блокноте это десять строк кода над результатами вывода.
- Для нарушения 6.2 цепочка доходит до трёх строк журнала и пункта положения: уровни объяснения из лекции 1.

Шаг 5. Неудовлетворимый класс

```
with onto:
    class СтатьяПоложение(Статья, Положение): pass
sync_reasoner_pellet(world)
СтатьяПоложение.equivalent_to   # [owl.Nothing]
```

- Классы Статья и Положение объявлены непересекающимися в лекции 2. Их общий подкласс не может иметь экземпляров.
- Машина сообщает об этом эквивалентностью классу Nothing. Данные для этого не нужны.
- В Protégé такой класс подсвечивается красным после запуска машины вывода.

Снимок окна Protégé: приложение Д.

Главное

Первая проверка онтологии после правки: запуск машины вывода на схеме. Красный класс есть ошибка проектирования.

Шаг 6. Граф форм

```
shp:Статья a sh:NodeShape ; sh:targetClass :Статья ;
  sh:property [ sh:path [ sh:inversePath :на_статью ] ;
    sh:qualifiedValueShape [ sh:property [ sh:path :сдано ; sh:minCount 1 ] ] ;
    sh:qualifiedMinCount 2 ; sh:message "6.1: сданных рецензий меньше двух" ] .

shp:Рецензия a sh:NodeShape ; sh:targetClass :Рецензия ;
  sh:property [ sh:path :на_статью ; sh:class :Статья ; sh:minCount 1 ; sh:maxCount 1 ] ;
  sh:property [ sh:path :рецензент ; sh:class :Рецензент ; sh:maxCount 1 ;
                sh:message "рецензентом должен быть рецензент" ] ;
  sh:property [ sh:path :шкала ; sh:in (5 10) ; sh:message "шкала 5 или 10" ] ;
  sh:property [ sh:path :сдано ; sh:lessThanOrEquals :срок ; sh:message "6.6: сдана позже срока" ] ;
  sh:sparql [ sh:message "6.2: рецензент из организации авторов" ; sh:select "..." ] .
```

- Четыре требования положения, одно предупреждение и три проверки записи. Файл `data/shapes.ttl`, около 50 строк.
- Каждая форма читается как фраза положения. Сообщение называет пункт.

Шаг 7. Отчёт о проверке

```
ok, report, text = validate(data, shacl_graph=shapes, inference="none")
Conforms: False | нарушений: 4 | предупреждений: 3
  P4              ← на_статью   6.1: сданных рецензий меньше двух         Violation
  P8              ← на_статью   6.1: сданных рецензий меньше двух         Violation
  рец_P3_Rv4      —             6.2: рецензент из организации авторов     Violation
  рец_P5_Rv3      сдано         6.6: рецензия сдана позже срока           Violation
  рец_P6_Rv4      ответ_авторов 6.5: ответ авторов не записан              Warning
  рец_P6_Rv5, рец_P8_Rv5        6.5: ответ авторов не записан             Warning
```

- Три требования, которые лекция 2 оставила без проверки, получили сообщения: 6.1 у двух статей, 6.6 у одной рецензии.
- Требование 6.2 найдено и правилом, и формой. Два средства сошлись.
- Предупреждения по 6.5 приём данных не останавливают, они попадают в отчёт: у P6 две рецензии без ответа, у P8 рецензия не сдана.

Главное

Отчёт называет узел, свойство и пункт положения. Это свидетельство, которое можно показать редактору.

Шаг 8. Строка с ошибкой: два средства рядом

Проверка по форме

```
рец_Р9_ошибка  рецензентом должен быть
               рецензент
  Result Path: :рецензент
  Value: :ФИЦ_ИУ
```

Форма сообщает сразу: значение не того класса. Одна строка, узел, свойство, значение.

Машина вывода

```
непротиворечива: False
  DisjointClasses(... Организация ... Рецензент)
  рец_Р9_ошибка рецензент ФИЦ_ИУ
  рецензент range Рецензент
  ФИЦ_ИУ type Организация
```

Машина выводит тип из области значений и находит противоречие с дизъюнктностью. Четыре строки.

- Ошибка одна, сообщения два. Форма проверяет запись, машина проверяет следствия.
- В проекте проверка по форме стоит первой: она дешевле и говорит прямо, какое поле исправить.

Шаг 9. Ответ редактора

статья	машина вывода	проверка данных	итог
P1	готова	нарушений нет	**выносить**
P2	готова	нарушений нет	**выносить**
P3	нарушает 6.2	6.2	нет
P4	---	6.1: одна сданная рецензия	нет
P5	готова	6.6: рецензия на 27-й день	нет
P6	---	6.5: ответ авторов не записан	нет
P7	готова	нарушений нет	**выносить**
P8	---	6.1: рецензий нет	нет

- Тот же ответ, что дали четыре правила в коде на лекции 1. Теперь знание о требованиях записано в онтологии и в формах, и каждое «нет» имеет пункт положения и строку отчёта.
- Машина вывода дала четыре кандидата, проверка данных сняла P5. Без любого из средств ответ неполон.

Главное

Ответ редактора собран из двух средств над одной записью. Он прослеживается до журнала и положения.

Часть 7

Архитектура агента: второй верификатор

Агент после лекции 2

	Компонент	В лекции 2
p_θ	модель	любая, сменная
D	ведущее приложение	цикл вызовов
T	инструменты	infer, graph_deps, check_consistency
P	контекст и инструкции	словарь классов в `AGENTS.md`
G	цель	критерий приёмки аксиомой или правилом
V	верификатор	машина вывода



Верификатор отвечал на вопрос «что следует из записанного». На вопрос «всё ли записано» агент отвечал «не подтверждено».

Главное

Агент лекции 2 честно молчал о требованиях 6.1 и 6.6. Сегодня он получает средство для них.

Инструмент проверки данных

```
validate_data()
  -> {"прошло": false,
      "нарушения": [
        {"узел": "P4", "требование": "6.1",
         "сообщение": "сданных рецензий меньше двух"},
        {"узел": "рец_P5_Rv3", "требование": "6.6",
         "сообщение": "сдана позже срока"}, ...]}

ready_articles()
  -> {"готовы": ["P1", "P2", "P5", "P7"],
      "основание": "класс ГотоваяКРешениюСтатья
                    после замыкания мира"}
```

Пометки ответа

источник	путь к записи корпуса
проверено	внешний верификатор
выведено	правило или аксиома
противоречие в данных	машина вывода
не прошло проверку	отчёт по форме, пункт положения
не подтверждено	сведений нет

Главное

Отчёт проверки возвращается агенту как результат инструмента. Каждое нарушение несёт пункт положения.

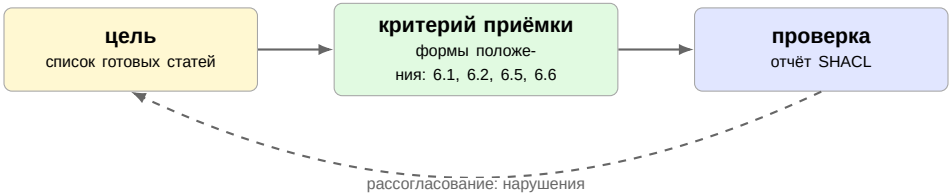
Прогон: какие статьи готовы

Вопрос: какие статьи можно выносить на решение редколлегии и почему остальные нельзя?

```
[1] модель вызвала check_consistency({})
    <- {"непротиворечива": true}
[2] модель вызвала validate_data({})
    <- {"прошло": false, "нарушения": [{"узел": "P4", "требование": "6.1", ...}, ... 5 записей]}
[3] модель вызвала ready_articles({})
    <- {"готовы": ["P1", "P2", "P5", "P7"], "основание": "класс ГотоваяКРешениюСтатья ..."}
[4] ответ модели:
Запись непротиворечива [найдено: check_consistency]. Готовы к решению P1, P2, P7 [выведено: класс
ГотоваяКРешениюСтатья] [проверено: validate_data]. P5 выведена как готовая, но её рецензия сдана позже
срока [не прошло проверку: 6.6]. P3 нарушает 6.2 [выведено: правило 6.2]. P4 и P8: сданных рецензий
меньше двух [не прошло проверку: 6.1]. P6: ответ авторов не записан [не прошло проверку: 6.5].
```

- Ответ редактора собран агентом из двух верификаторов. У каждой части пометка.
- Инструкция агента: статья считается готовой, если она выведена готовой и не имеет нарушений в отчёте.

Критерий приёмки как форма



- В лекции 2 критерий приёмки записывался аксиомой или правилом. Сегодня он записывается формой: ответ принимается, когда данные, на которых он построен, прошли проверку.
- Рассогласование возвращается агенту списком нарушений с пунктами положения. Агент исключает статьи из ответа и называет причину.
- Акцептор результата по Анохину получил проверяемую запись: форму можно прочитать до действия и применить после.

Главное

Форма есть критерий приёмки, записанный на языке данных. Это самый прямой способ задать агенту, что считается результатом.

Что изменилось в компонентах

Компонент	Что добавила лекция 3
контекст P	компетентностные вопросы записаны в `AGENTS.md`: агент знает, на какие вопросы онтология отвечает и каким средством
инструменты T	`validate_data` возвращает отчёт по формам, `ready_articles` возвращает выводимый класс после замыкания
верификатор V	два средства: машина вывода для следствий, проверка по форме для полноты и значений
цель G	критерий приёмки записан формой и читается до действия
модель p_θ	без изменений

Главное

Онтология, спроектированная по вопросам, и формы, записанные по требованиям, делают верификатор агента полным для задачи редактора. На лекции 4 агент получит запросы к графу.

Часть 8

Задание

Что сдать

Задание лекции 3 --- третья часть задания модуля 1. Оно выполняется на онтологии лекции 2 и пяти вопросах исследователя лекции 1.

Мини-практика: повторить блокнот лекции, запустить проверку по формам на `review.owl`, внести ошибку в данные и прочитать отчёт.

Артефакт	Содержание	Критерий приёмки
`kg/QUESTIONS.md`	3--5 компетентностных вопросов, у каждого ожидаемый ответ, источник и средство, таблица «вопрос --- класс --- свойство»	каждый класс нужен хотя бы одному вопросу
`kg/ontology.ttl`	версия 2: не меньше одного выводимого класса по вопросу, проверка иерархии по фразе	машина вывода без противоречий, нет неудовлетворимых классов, выведен хотя бы один факт
`kg/shapes.ttl`	не меньше трёх форм: число связей, тип или допустимые значения, сравнение или составная форма	проверка проходит на данных без ошибки и ловит намеренно испорченную запись
`kg/VALIDATION.md`	лог машины вывода, отчёт проверки до и после исправления	оба отчёта приложены
`diary.md`, `AGENTS.md`	запись о работе, вопросы в разделе «чего агент не знает сам»	заполнены

Образец: вопросы и формы

kg/QUESTIONS.md

```
## Вопросы
| № | Вопрос | Ответ | Средство |
| 1 | Какие теоремы доказаны? | A14, A10 | выводимый класс |
| 2 | На что опирается A14?  | 11 утв. | транзитивное свойство |
| 3 | У каких утверждений нет
      доказательства?       | C3      | форма SHACL |

## Вопрос --- класс
| вопрос | классы | свойства |
| 1 | Теорема, ДоказаннаяТеорема | доказывается |
```

kg/shapes.ttl

```
shp:Теорема a sh:NodeShape ;
  sh:targetClass :Теорема ;
  sh:property [ sh:path :доказывается ;
    sh:minCount 1 ; sh:class :Доказательство ;
    sh:message "у теоремы нет доказательства" ] ;
  sh:property [ sh:path :зависит_от ;
    sh:minCount 1 ;
    sh:message "теорема ни на что не опирается" ] .
```

Пример по математике. Та же структура для стандарта, учебника или набора статей.

Образец: отчёт проверки

kg/VALIDATION.md

```
## Машина вывода
Неудовлетворимых классов: нет.
Выведено: ДоказаннаяТеорема = {A14, A10}.
Основание: определение класса и факты доказывается.

## Проверка по формам, данные без ошибки
Conforms: True. Нарушений: 0.

## Проверка, намеренная ошибка
Внесено: у теоремы A10 удалена связь доказывается.
Conforms: False.
  A10  у теоремы нет доказательства  [sh:minCount]
Исправлено: связь возвращена, Conforms: True.
```

- Отчёт приводится в том виде, в каком его выдал pySHACL. Сокращать можно, переписывать нельзя.
- Намеренная ошибка вносится в копию данных. Сдаваемый файл остаётся чистым.
- Выведенный факт отличается от записанных: сверка списка до и после вывода.

Критерии и типичные ошибки

Содержательные критерии

- У каждого вопроса есть средство и ожидаемый ответ.
- Выводимый класс отвечает на вопрос, и машина наполняет его.
- Формы читаются как требования области и называют их в сообщениях.
- Проверка ловит испорченную запись и проходит на чистой.

Формальные критерии проверяет `scripts/check_l3.py`: разделы файлов, число форм, запуск pySHACL, запуск машины вывода.

Типичные ошибки

- Форма на выведенное свойство: обратное или транзитивное свойство в `sh:path` без пути.
- Выводимый класс с «все» или «нет» без замыкания мира: класс пуст, вывод «не работает».
- Вопрос без ожидаемого ответа: его нельзя проверить.
- Сообщение формы без пункта требования.
- Намеренная ошибка внесена в сдаваемый файл.

Упражнения

1. Записать три вопроса своей области выражениями классов. Для каждого назвать денотат.
2. Найти ошибки иерархии в заготовке из восьми классов: часть, роль, индивид, класс по признаку.
3. Написать форму для фразы «у каждого утверждения есть доказательство» и форму для «оценка от 0 до 10».
4. Прочитать отчёт проверки из шести нарушений и назвать для каждого пункт положения и поле.
5. Записать требование 6.2 тремя способами: правило, выражение класса, форма.
6. Объяснить, почему форма для 6.1 нуждается в обратном пути, и что изменится при включении вывода перед проверкой.
7. Для одной ошибки данных получить сообщение формы и объяснение машины вывода и сравнить их.

Ответы даны в конце файла упражнений.

Главное

Упражнения служат для закрепления и не оцениваются. Статья недели: W3C, «SHACL», разделы 1--2 и 4.1--4.3 рекомендации.

Итог лекции 3

1. Онтология проектируется от компетентностных вопросов. Вопрос с ожидаемым ответом есть тест онтологии.
2. Три вида классов: простой задаёт иерархию, связанный задаёт обязательство, выводимый задаёт вопрос. Вопрос «какие объекты» записывается выводимым классом.
3. Иерархия проверяется фразой «каждый A является B». Пять ошибок: часть, роль, индивид, значение признака, склейка.
4. Машина вывода классифицирует схему и индивидов и объясняет каждый вывод. Неудовлетворимый класс есть ошибка проектирования. Выводимые классы с «все» и «нет» наполняются после замыкания мира.
5. SHACL проверяет данные в замкнутом мире: число связей, типы, значения, сравнения. Форма видит только записанное.
6. Одно требование положения получает четыре записи: правило, класс, форма, запрос. Выбор зависит от того, что нужно дальше.
7. Агент получил второй верификатор и пометку «не прошло проверку». Ответ редактора собран из двух средств.

Главное

Машина вывода отвечает, что следует из записанного. Проверка по форме отвечает, всё ли записано и верно ли. Онтология, спроектированная по вопросам, нужна обоим.

К следующему занятию

Лекция 4. Графы знаний: RDF, SPARQL и наполнение из текста. Онтология и данные становятся графом троек с точкой доступа. Вопросы 2 и 4 редактора получают запросы: путь по свойствам и соединение по организации. Граф наполняется извлечением из текста языковой моделью.

- **Статья недели:** W3C, «SHACL», разделы 1--2 и 4.1--4.3. Вопрос: какое требование своей области записывается только составной формой или запросом.
- **Принести:** `kg/shapes.ttl` с тремя формами и отчёт проверки на своих данных.
- **Установить:** `rdflib` с поддержкой SPARQL и одно хранилище троек: Oxigraph или Fuseki.
- **Сделать:** записать один вопрос исследователя, который требует соединения двух сущностей через третью.

Литература I

1. Gruber T. R. A translation approach to portable ontology specifications. Knowledge Acquisition, 5(2), 1993.
2. Studer R., Benjamins V. R., Fensel D. Knowledge engineering: principles and methods. Data & Knowledge Engineering, 25, 1998.
3. Arp R., Smith B., Spear A. D. Building Ontologies with Basic Formal Ontology. MIT Press, 2015. Главы 1--3, 5.
4. ISO/IEC 21838-2:2021. Information technology. Top-level ontologies. Part 2: Basic Formal Ontology.
5. W3C. PROV-O: The PROV Ontology. W3C Recommendation, 2013.
6. Fernández-López M., Gómez-Pérez A., Juristo N. METHONTOLOGY: From ontological art towards ontological engineering. AAAI Spring Symposium, 1997.
7. Grüninger M., Fox M. S. Methodology for the design and evaluation of ontologies. IJCAI-95 Workshop on Basic Ontological Issues in Knowledge Sharing, 1995.
8. Guarino N., Welty C. An overview of OntoClean. In: Handbook on Ontologies, Springer, 2009.
9. W3C. Shapes Constraint Language (SHACL). W3C Recommendation, 2017.
10. Knublauch H., Kontokostas D. SHACL: разделы 2--4 рекомендации. Ядро, цели, компоненты ограничений.
11. Noy N. F., McGuinness D. L. Ontology Development 101: A Guide to Creating Your First Ontology. Stanford KSL, 2001.
12. Rector A. et al. OWL Pizzas: Practical Experience of Teaching OWL-DL: Common Errors and Common Patterns. EKAW 2004.

Литература II

13. Krötzsch M., Simančík F., Horrocks I. A Description Logic Primer. arXiv:1201.4089, 2012.
14. Baader F., Horrocks I., Lutz C., Sattler U. An Introduction to Description Logic. Cambridge University Press, 2017. Главы 2, 4.
15. Horridge M. A Practical Guide to Building OWL Ontologies Using Protégé 4 and CO-ODE Tools. 2011. Главы 4, 6.
16. Lamy J.-B. Ontologies with Python. Apress, 2021. Главы 7--8: машина вывода, замыкание мира.
17. Sirin E. et al. Pellet: A practical OWL-DL reasoner. Journal of Web Semantics, 5(2), 2007.
18. Horridge M., Parsia B., Sattler U. Laconic and precise justifications in OWL. ISWC 2008. Объяснения вывода.
19. Reiter R. On closed world data bases. In: Logic and Data Bases, Plenum, 1978.
20. Gayo J. E. L., Prud'hommeaux E., Boneva I., Kontokostas D. Validating RDF Data. Morgan & Claypool, 2018. Главы 4--5: SHACL.
21. Sommer A. pySHACL. Документация и исходный код, 2024.
22. Hogan A. et al. Knowledge Graphs. ACM Computing Surveys, 54(4), 2021. Раздел 3.1: схема и формы.
23. Анохин П. К. Очерки по физиологии функциональных систем. М.: Медицина, 1975.
24. Keet C. M. An Introduction to Ontology Engineering. 2018. Главы 5--6: методологии, компетентностные вопросы.

Приложение А. Шпаргалка SHACL

Что хотим проверить	Конструкция	Пример
ко всем индивидам класса	`sh:targetClass :A`	`sh:targetClass :Рецензия`
свойство записано	`sh:minCount 1`	у рецензии есть статья
не больше одного значения	`sh:maxCount 1`	один рецензент
значение есть индивид класса	`sh:class :B`	рецензентом должен быть рецензент
тип литерала	`sh:datatype xsd:date`	дата сдачи есть дата
границы числа или даты	`sh:minInclusive`, `sh:maxInclusive`	оценка от 0
список значений	`sh:in (5 10)`	шкала 5 или 10
вид строки	`sh:pattern` с регулярным выражением	идентификатор требования
сравнение двух свойств	`sh:lessThanOrEquals :срок`	сдано не позже срока
свойство в обратную сторону	`sh:path [ sh:inversePath :p ]`	рецензии статьи
путь из двух шагов	`sh:path (:p :q)`	организация авторов рецензии
сколько значений нужного вида	`sh:qualifiedValueShape`, `sh:qualifiedMinCount`	две сданные рецензии
условие запросом	`sh:sparql [ sh:select "..." ]`	требование 6.2
строгость	`sh:severity sh:Warning`	6.5 как предупреждение
текст в отчёте	`sh:message "..."`	пункт положения

Запуск: `pyshacl -s shapes.ttl data.ttl` или `validate(data, shacl_graph=shapes, inference="none")`.

Приложение Б. Словарь лекции

Термин	Значение	Термин	Значение
онтология	спецификация концептуализации области: классы, свойства, аксиомы	граф данных	индивиды и факты, которые проверяются
онтология верхнего уровня	общие классы для всех областей: BFO, DOLCE	граф форм	требования к данным на языке SHACL
компетентностный вопрос	вопрос, на который онтология должна отвечать	узловая форма	набор требований к узлам одного вида
простой класс	задан местом в иерархии	свойство формы	требование к одному свойству узла
связанный класс	задан необходимым условием со связью	путь	свойство или цепочка свойств в форме
выводимый класс	задан необходимым и достаточным условием	составная форма	подсчёт значений нужного вида
классификация	машина строит иерархию и относит индивидов к классам	отчёт	граф нарушений: узел, путь, сообщение
неудовлетворимый класс	класс без возможных экземпляров	строгость	нарушение, предупреждение, замечание
объяснение	аксиомы и факты, из которых получен вывод	открытый и замкнутый мир	отсутствие факта: ничего не значит или нарушение
замыкание мира	заявление о полноте записи для индивида	роль	нежёсткий класс: объект может его покинуть

Приложение B. Вопросы для самопроверки

1. Что делает вопрос компетентностным? Запишите один для своей области.
2. Чем выводимый класс отличается от связанного? Приведите по одному из задачи редактора.
3. Почему класс ГотоваяКРешениюСтатья пуст до замыкания мира?
4. Что означает эквивалентность класса Nothing после вывода?
5. Назовите пять ошибок иерархии и проверку, которая ловит каждую.
6. Чем `sh:minCount 1` отличается от ограничения существования в OWL?
7. Почему форма для 6.1 написана через `sh:inversePath`?
8. Как записать требование 6.6, если в ядре SHACL нет арифметики дат?
9. В каких четырёх записях выражено требование 6.2 и что даёт каждая?
10. Что проверяет форма и что она не проверяет?
11. Какую пометку получает часть ответа агента, построенная на данных с нарушением?
12. Что ответит машина вывода и что ответит форма на запись организации в поле «рецензент»?

Приложение Г. Вариант по математике

Корпус планиметрии, 30 утверждений, статья P2 с утверждениями C1--C3. Скелет тот же: утверждение, обоснование, зависимость.

вопрос	какие теоремы доказаны и на что опираются
выводимый класс	ДоказаннаяТеорема $\equiv$ Теорема $\sqcap \exists$доказывается. (Доказательство $\sqcap$ вердикт $=$ принято)
форма	у каждой теоремы есть доказательство, у каждого утверждения есть зависимость

Что показывает блокнот, шаг 11

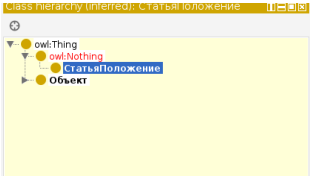
- Машина вывода относит C1 и C2 к доказанным, C3 остаётся вне класса: вердикт внешней проверки «отклонено».
- Форма «у утверждения есть доказательство» проходит. Форма «вердикт принят» даёт нарушение на C3.
- Те же два средства, та же роль: класс отвечает «что следует», форма отвечает «что записано».

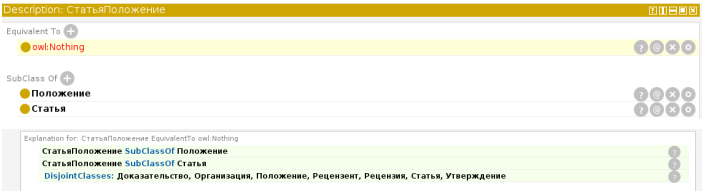
Математика здесь служит примером области с внешней проверкой. В вашей области им будет тест или сверка с данными.

Вопросы

oataeva@frccsc.ru khalov.a@phystech.edu

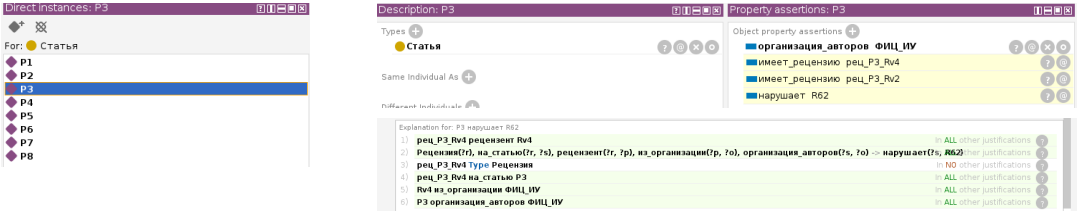
Приложение Д. Неудовлетворимый класс в Protégé





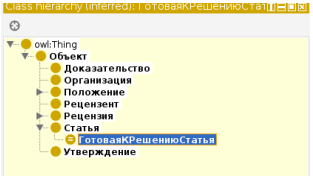
Файл `data/review_unsat.owl`, Pellet. В выведенной иерархии класс СтатьяПоложение стоит под `owl:Nothing` красным, в описании на жёлтом фоне выведено `Equivalent To owl:Nothing`. Объяснение называет две аксиомы подкласса и дизъюнктность.

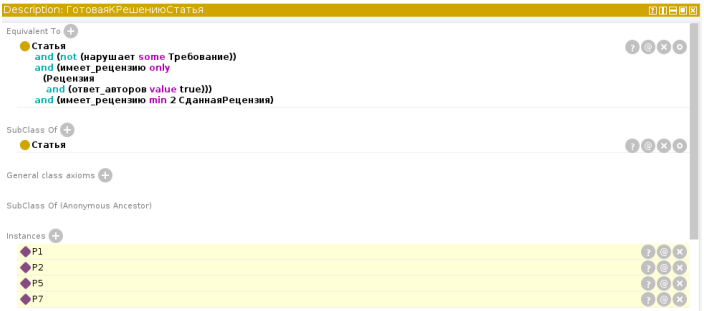
Приложение Д. Выведенный факт и его объяснение



Файл `data/review.owl`, статья P3. На жёлтом фоне выведенное: две обратные связи имеет_рецензию и факт нарушает R62. Объяснение: правило 6.2 и факты о рецензии Rv4 и организации ФИЦ ИУ.

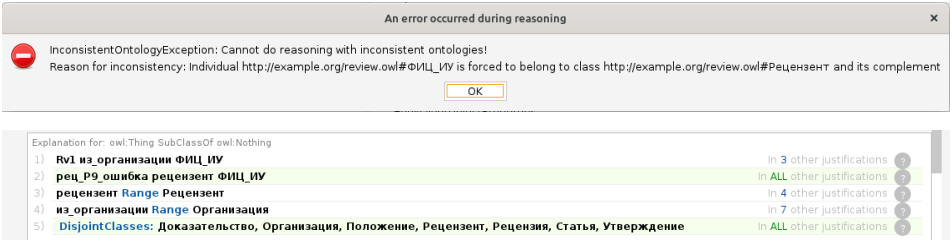
Приложение Д. Выводимый класс после замыкания





Файл `data/review_v2.owl`: определение класса ГотоваяКРешениюСтатья и замыкание мира у статей. Pellet относит к классу P1, P2, P5, P7, экземпляры показаны на жёлтом фоне.

Приложение Д. Противоречие и его объяснение



Файл `data/review_error.owl`: рецензентом рецензии рец_P9_ошибка записана организация. Pellet останавливается: индивид ФИЦ_ИУ должен быть рецензентом и не рецензентом сразу. Объяснение называет факт, области значений двух свойств и дизъюнктность классов.